



Stabilizing LinkedIn Continuous Deployment on Airflow



Wensi Hu, Pooja Pal @LinkedIn

Meet the Team



**"Hi. I keep LinkedIn's deployments safe and fast on Airflow.
Off the clock: hiking, cooking, gym, tennis."**

Wensi Hu | Sr. Software Engineer

**"Hi, I build the reliable, automated Airflow infra that lets
thousands of engineers ship every day.**

**In my spare time: I enjoy visiting new food joints, walk miles
in the nature."**

Pooja Pal | Software Engineer



The scale we keep stable

- 10K+ engineers deploying every day

- 6,000+ services plus ML pipelines

- 5,000+ DAGs and 100K+ tasks per day

- Largest pipeline ~800 tasks



"3 years, 6,000+ services, 100K deploys a day - and it just works."

"...most days. Let's talk about the days it doesn't. 😊"



From scaling to stabilizing

Last Year

Platform Build

Our journey building a unified, touchless CD platform on Airflow.

This Year

Scale & Speed

Keeping the continuous deployment platform fast and reliable at scale.

Core Focus

Key Pillars

- Parallel rollbacks
- Pipeline consistency
- Cross-cluster failover
- Deep observability & learnings

Five requirements, five layers

Self-service at
scale

UI/UX

10K engineers trigger and track deployments.

Security and
correct routing

Gateway

Authenticates and routes each pipeline to the right Airflow cluster.

High availability
and scale

Orchestration

Airflow clusters across regions for redundancy.

Safety and
governance

Execution + policy

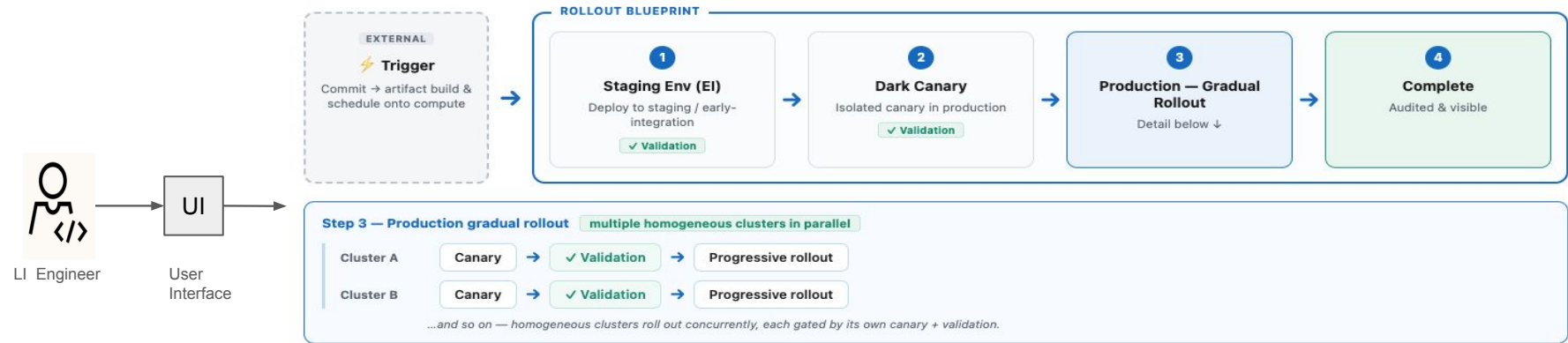
Each deploy action runs as an isolated step
(**KubernetesPodOperator**),
cleared by the policy engine
before it executes.

Visibility and
fast recovery

Observability

Structured and unstructured signals for the full picture

End User Touch Points



Deployment-Intent.yaml

```
targets:
- clusters:
  - ei-████.DEFAULT
  - ei4.DEFAULT
  - prod-████.DEFAULT
  - prod-████.commerce-quoting.dark
  - prod-████.DEFAULT
  - prod-████.commerce-quoting.dark
  - prod-████.DEFAULT
  - prod-████.commerce-quoting.dark
types:
- k8s
deployable_name: commerce-quoting
```

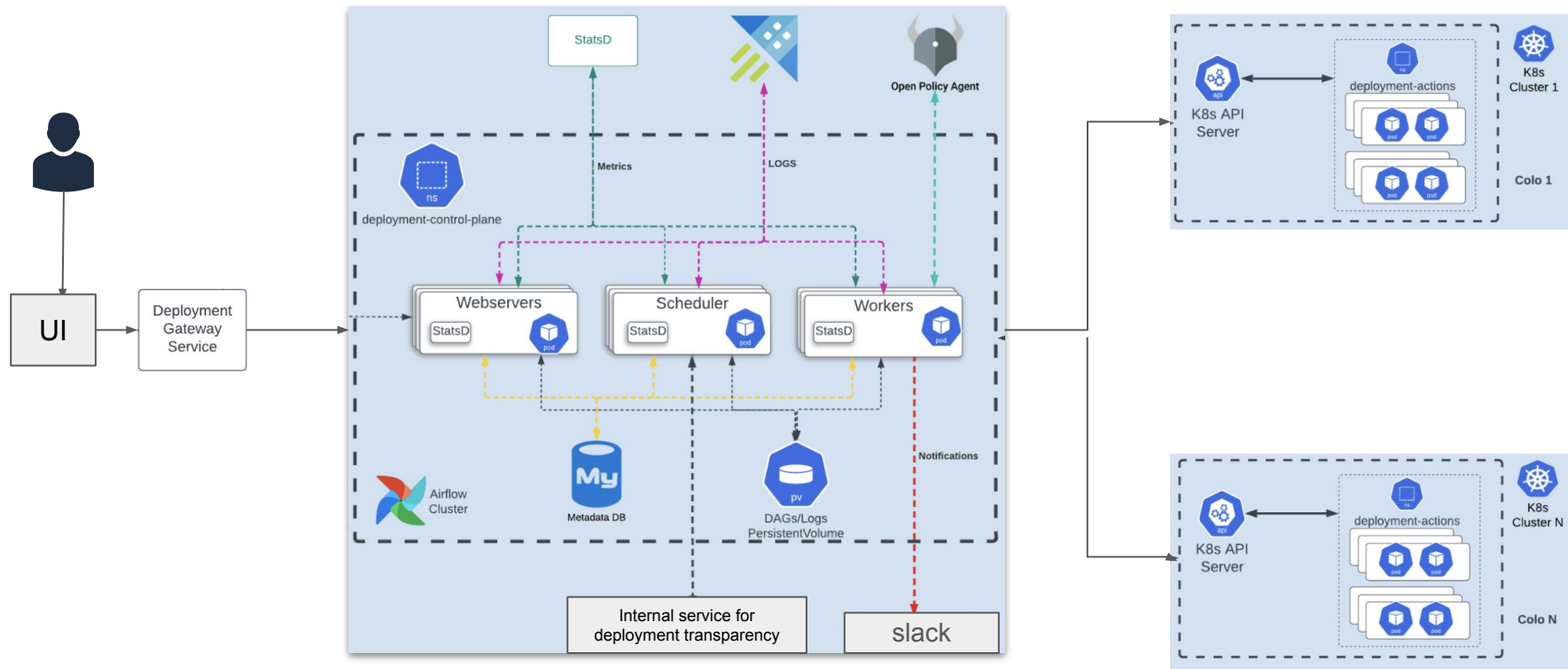
Deployment Pipeline

→ **DAG generator component**



Low-code or no-code design

Architecture Under the hood



Design Principles

01

Move validations to Left

- Lesser error in later deployment stages
- Prevent incorrect changes before merging
- Static Analysis can validate yaml schema and check for required settings

02

Pipeline steps are loosely coupled with workflow

- Steps are language agnostic
- Some steps are containerized deployment actions while others are simple python ops.
- Democratise the Platform for Partners

03

Human In the Loop is necessary

- Pause and Validate Action
- Manual Overrides
- Configurable soak steps

Serial for safety, parallel for speed



Normal deploys roll out stage by stage, safest environment first, with validation gates between.



Rollbacks and emergency deploys need the same targets, but fast and in parallel.



A serial pipeline is inefficient here: each step spins up a fresh `KubernetesPodOperator` worker pod.



So parallel rollout runs on its own fast path over the same targets.

Dynamic Task Mapping: Airflow for Fast Rollback

Leveraging native dynamic task mapping to orchestrate parallel target rollbacks and efficient partial reruns.

Core Mechanism

01 Dynamic Expansion

Airflow's built-in dynamic task mapping expands one mapped task into N at runtime based on active targets.

02 Parallel Fan-Out

We map over deployment targets so N target tasks fan out and execute in parallel simultaneously.

03 Zero Pipeline Overhead

Operates within a single DAG without secondary pipeline overhead, staying fully inside dynamic-DAG limits.

Targeted Recovery & Flow

• Emergency Rollback

Act immediately and exactly on the targets we choose, executing all rollbacks in parallel.

• Smart Rerun Strategy

Clear and re-map only the specific failed targets instead of restarting the entire DAG.

Execution Pipeline

```
get_targets → .expand() → deploy(t1..N) in  
parallel → join
```



Pipeline Consistency

1

What is it?

Stable DAG definition throughout a pipeline run

2

Why is it needed?

Prevent execution and UI inconsistencies

3

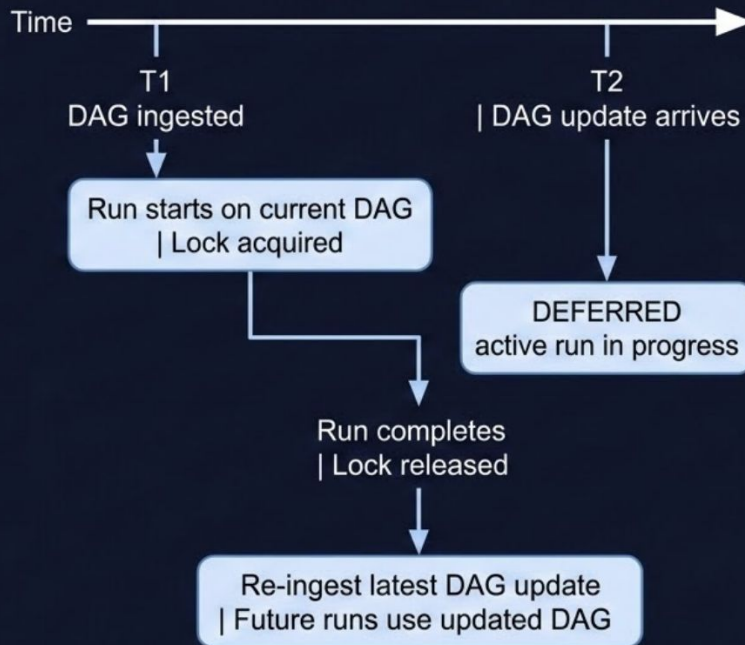
Challenge

Pipeline definitions can change while runs are active

4

Our Approach

Defer updates until completion and re-ingest safely



Pipeline Versioning

01 Core Correctness Paradigm

Pipeline consistency solves active-run correctness, while pipeline versioning solves historical correctness.

02 Versioned DAG Ingestion

Every structural DAG change creates a new pipeline version persisted in the metadata store.

03 Traceable Execution History

Each run is mapped to the pipeline version that generated it, enabling accurate lineage reconstruction.

04 Version-Aware User Experience

Run → Pipeline Version → Execution Lineage

Multi-Cluster Routing & Failover

Scaling to 5,000+ DAGs with automated gateway routing and high-availability workload isolation.

From (Strained Single Cluster)

A single shared cluster became a blast-radius risk at **5,000+ DAGs**

Noisy or critical workflows interfere and degrade shared resources

Cluster outage blocks progress and takes down all pipelines

Active cluster fails to launch pods, halting scheduled execution

Static config-based manual routing and failover decisions



To (Resilient Multi-Cluster)

Same code runs seamlessly on **multiple Airflow clusters** across data centers

Gateway routing spreads execution load and isolates critical workloads

Re-route pipelines in bulk quickly with a single, small config change

Failover rescue: instantly re-routed running tasks to a healthy cluster

Next Step: fully dynamic, automatic cluster routing

Observability: catch the gap before users do

1

Structured and unstructured signals from every task and pod

2

Track deployment success and latency against a platform SLO

3

Alert on leading signals: pod-launch failures, tasks stuck In Progress, lingering pods, scheduler heartbeat

4

Goal: cut detection time from hours to minutes

5

Playbooks turn alerts into fast, repeatable mitigations

Incident Analysis

A stale pod resurrects and re-deploys an old version of the application.

- 
- 01 Each deploy job runs as a K8s job and pod orchestrated from Airflow
 - 02 A node drain left a job's pod stuck terminating; weeks later the job spawned a new pod
 - 03 That new pod re-ran the OLD deployment intent, shipping an outdated version
 - 04 Found by a user ~30 hours later, not by an alert. The gap was detection
 - 05 **Fix:** time out orchestration jobs and pods; alert on lingering pods

Key Takeaways

01

Airflow can successfully power large-scale deployment orchestration beyond traditional data pipelines.

02

Extend native capabilities to fit your use case.

03

Abstract complexity from end users.



04

Reliability must be designed into the platform.

Thank you. 🤝

